==========

==========

CHAPTER 6 — STAGE-1 TRANSLATION (VA → IPA)

==========

==========

A maximum-detail, RTOS/QNX-style technical breakdown of how ARMv8 performs Stage-1 translation, including descriptor interpretation, permission enforcement, attribute merging, and IPA generation.

****

****

6.1 — Introduction to Stage-1 Translation

****

****

Stage-1 translation is performed by the **guest OS** (EL1). Its purpose is to convert:

```
Virtual Address (VA) → Intermediate Physical Address (IPA)
```

Stage-1 defines:

- guest virtual memory
- process isolation
- user/kernel separation
- guest permissions
- guest memory attributes
- guest shareability

Stage-1 does **not** produce real physical addresses. It produces **IPA**, which is later translated by Stage-2.

****

****

6.2 — Stage-1 Translation Regime

****

****

Stage-1 applies to:

- EL0 (user mode)
- EL1 (kernel mode)

It is controlled by:

- **TTBR0_EL1** (user space)
- **TTBR1_EL1** (kernel space)
- **TCR_EL1** (translation configuration)
- **MAIR_EL1** (memory attributes)

Teaching Note

Stage-1 is entirely under OS control. Hypervisor cannot directly modify Stage-1 tables.

****

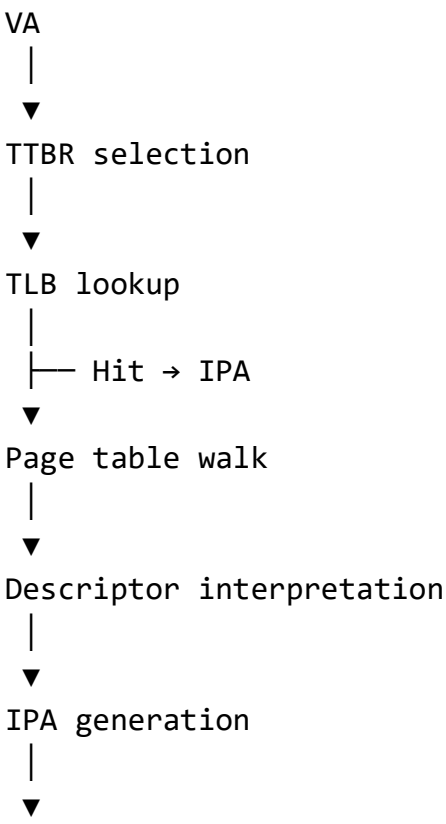
****

6.3 — Stage-1 Translation Flow

****

****

High-level flow:



```
Insert into TLB
```

****

****

6.4 — TTBR Selection

****

****

The MMU selects TTBR based on VA:

```
if VA in TTBR0 region:
    use TTBR0_EL1
else:
    use TTBR1_EL1
```

TTBR0_EL1:

- per-process
- ASID-tagged
- user space

TTBR1_EL1:

- global
- kernel space
- shared across processes

****

****

6.5 — Stage-1 TLB Lookup

****

****

TLB entries are tagged with:

- VA
- IPA
- ASID
- permissions
- attributes

TLB lookup requires:

```
VA + ASID match
```

If hit:

- permissions checked
- attributes applied
- IPA returned

If miss:

- page table walk begins.

****

****

6.6 — Stage-1 Page Table Walk

****

****

Walk steps:

1. Read L1 descriptor
2. If table → read L2
3. If table → read L3
4. If page/block → final mapping
5. Extract fields
6. Produce IPA

Walk cache accelerates steps 1–3.

Teaching Note

Walk cache stores intermediate descriptors.

****

****

6.7 — Descriptor Interpretation

****

****

Stage-1 descriptors define:

- physical base address (IPA base)
- AP (access permissions)
- SH (shareability)
- AttrIndx (memory attributes)
- AF (access flag)
- PXN/UXN (execute permissions)

Implementation Note

Descriptor interpretation is identical for Stage-1 and Stage-2.

****

****

6.8 — Access Flag (AF) Handling

****

****

AF indicates whether the page has been accessed.

- **AF = 1** → page is accessible
- **AF = 0** → access generates Access Flag Fault

OS typically sets AF=1 for all pages.

****

****

6.9 — Access Permission (AP) Enforcement

****

****

AP controls read/write permissions.

AP	Meaning
00	EL0/EL1 read/write
01	EL1 read/write, EL0 no access

10 EL0/EL1 read-only
11 EL1 read-only, EL0 no access

Teaching Note

AP interacts with PAN and SCTLR.WXN.

****

****

6.10 — Execute Permission Enforcement (PXN/UXN)

****

****

PXN → EL1 cannot execute UXN → EL0 cannot execute

Used to enforce:

- W^X
- kernel non-exec
- user non-exec

Implementation Note

PXN/UXN are critical for security.

****

****

6.11 — Shareability Enforcement (SH)

****

****

SH controls cache coherence.

SH	Meaning
00	Non-shareable
10	Outer Shareable
11	Inner Shareable

Teaching Note

Inner Shareable required for SMP.

****

****

6.12 — Memory Attribute Selection (AttrIndx → MAIR_EL1)

****

****

AttrIndx selects MAIR entry.

Example:

```
AttrIndx = 3 → MAIR_EL1[3]
```

Memory attributes define:

- cacheability
- ordering
- device behavior

****

****

6.13 — IPA Generation

****

****

IPA is computed as:

```
IPA = PA_base + page_offset
```

Where:

- PA_base = descriptor[47:12]
- page_offset = VA[11:0]

Teaching Note

IPA is not the real physical address.

****

****

6.14 — Stage-1 Permission Faults

****

****

Faults include:

- Access Flag Fault
- Permission Fault
- Execute Fault
- Alignment Fault
- Translation Fault

Implementation Note

Faults occur at earliest possible stage.

****

****

6.15 — Stage-1 Translation Example (Full Walkthrough)

****

****

Given:

`VA = 0x0000_7FFF_1234_5678`

Step 1 — TTBR selection

TTBR0_EL1.

Step 2 — TLB lookup

Miss.

Step 3 — L1 descriptor

Type = table → go to L2.

Step 4 — L2 descriptor

Type = block → final mapping.

Step 5 — Extract fields

- PA_base = 0x0000_2000_0000_0000

- AP = EL0/EL1 read/write
- SH = Inner Shareable
- AttrIndx = 0 (Normal WB)
- UXN = 1

Step 6 — IPA generation

IPA = PA_base + offset.

****

****

6.16 — Stage-1 and Kernel/User Separation

****

****

TTBR0_EL1:

- user space
- ASID-tagged
- per-process

TTBR1_EL1:

- kernel space
- global
- shared across processes

Teaching Note

Kernel mappings remain in TLB across context switches.

****

****

6.17 — Stage-1 and Speculation

****

****

Speculative execution may:

- read descriptors
- fill TLB entries
- perform speculative walks

Implementation Note

Use barriers when updating page tables.

****

****

6.18 — Stage-1 and Memory Ordering

****

****

Page table updates must follow:

1. Write descriptor

2. DSB
3. TLBI
4. DSB
5. ISB

Teaching Note

Skipping barriers causes silent corruption.

****

****

6.19 — Stage-1 and QNX/RTOS Behavior

****

****

QNX uses Stage-1 for:

- process partitions
- system partition
- I/O partition
- memory protection
- ASID-based isolation

Implementation Note

QNX may use static ASID ranges.

****

****

6.20 — Chapter Summary

****

****

This chapter covered:

- Stage-1 translation flow
- TTBR selection
- TLB lookup
- page table walk
- descriptor interpretation
- AF, AP, PXN/UXN, SH, AttrIndx
- IPA generation
- fault types
- full examples
- memory ordering
- QNX/RTOS considerations